

Intelligence Leading the Frontier, Computing Shaping the Future HIT Code Review Research

Experiment Report Mining Code Review Comments and Understanding Requirements

Name	Ulugbek Khamidov, Komil Tokhirov	Department	HITCodeReviewResearch
Student ID	2026140348, 2026140354	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	HIT First Campus
Lab Time	2024-06-11 14:00-17:00		
Lab Score		Discussion Score	
Report Score		Total Experi- ment Score	

1. Experiment Objectives

- Understood traditional machine learning workflow for code review. - Mastered data preprocessing for code review tasks. - Extracted AST and CFG representations from code. - Extracted structural, textual, and statistical features. - Trained SVM and Random Forest models. - Built a merge prediction model and analyzed performance.

2. Experiment Content

- Filtered 1,020 human-written PRs from Experiment 1 dataset. - Extracted AST representations using Python's ast module. - Extracted CFG representations for control flow analysis. - Extracted 9 features: additions, deletions, changed_files, commits_count, reviewers_count, comments_count, labels_count, pr_length, has_ai_reviewer. - Constructed ML dataset: 816 train, 204 test. - Trained SVM (RBF kernel) and Random Forest (100 trees). - Evaluated using accuracy, precision, recall, F1, ROC-AUC.

3. Experimental Results

We trained SVM and Random Forest on 1,020 human-written PRs (601 merged, 419 not merged).

SVM Results: - Accuracy: 88.7% - Precision: 88.2% - Recall: 93.3% - F1: 90.7% - ROC-AUC: 92.3%

Random Forest Results: - Accuracy: 91.2% - Precision: 88.6% - Recall: 97.5% - F1: 92.9% - ROC-AUC: 97.3%

Random Forest outperformed SVM across all metrics. Feature importance analysis showed labels_count (45.3%) as the dominant predictor, followed by pr_length (12.2%) and deletions (10.3%).

AST and CFG examples were generated for sample code snippets.

Figures

AST Examples

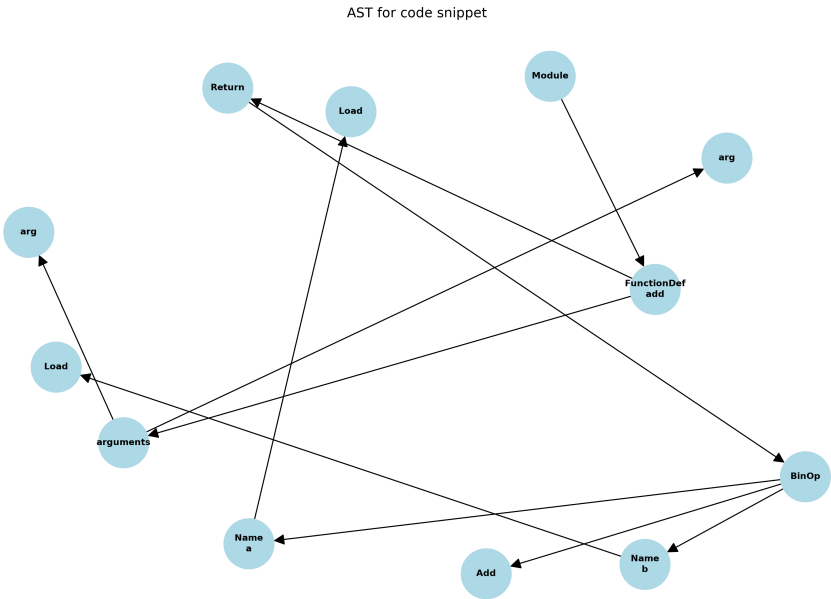


Figure 1: AST for a simple addition function

CFG Examples

Model Performance

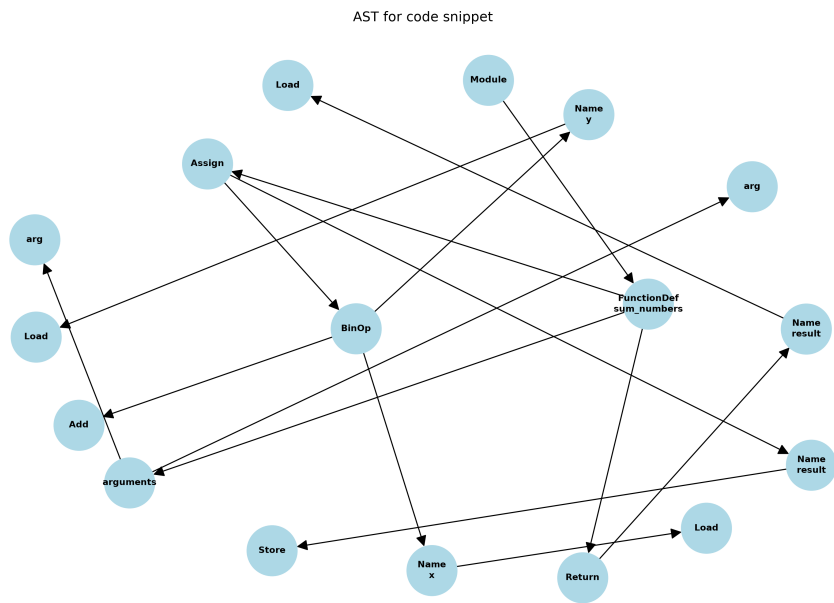


Figure 2: AST for a function with intermediate variable

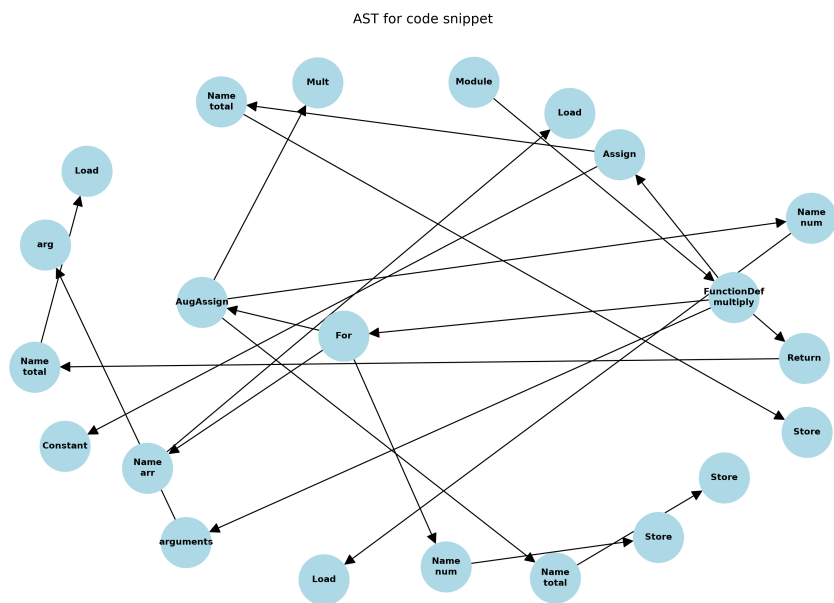


Figure 3: AST for a function with loop

CFG for code snippet

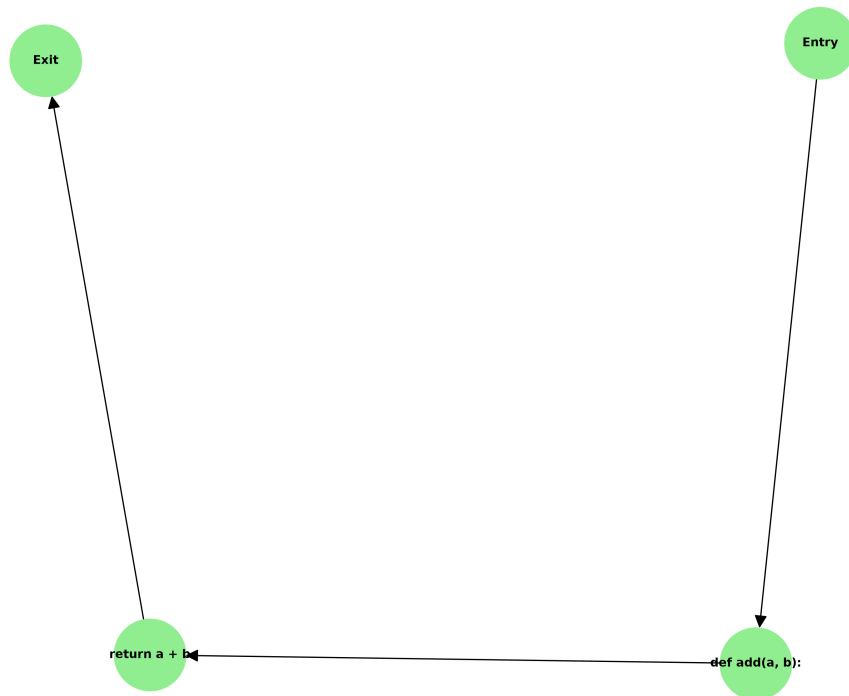


Figure 4: CFG for a simple addition function

CFG for code snippet

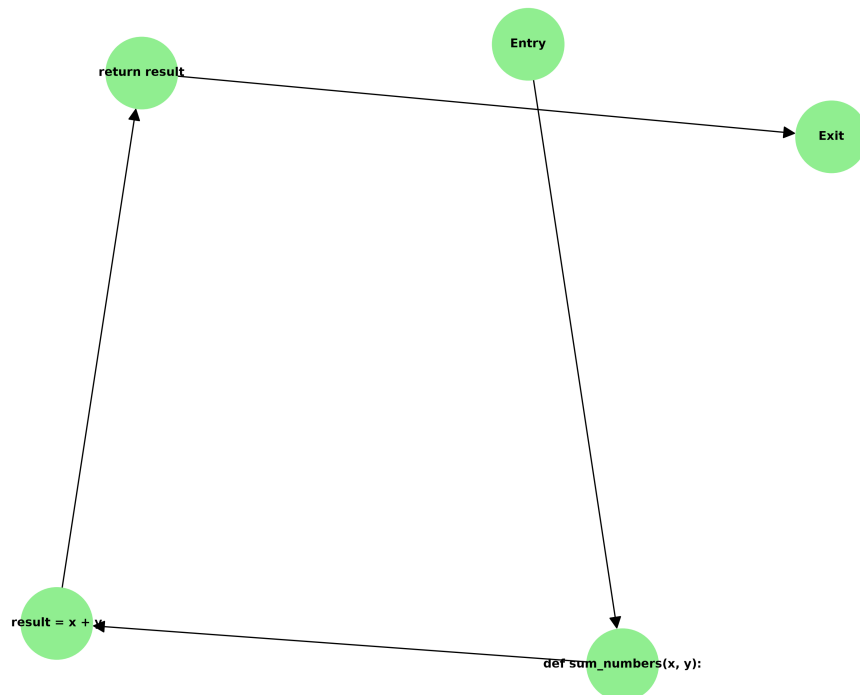


Figure 5: CFG for a function with intermediate variable

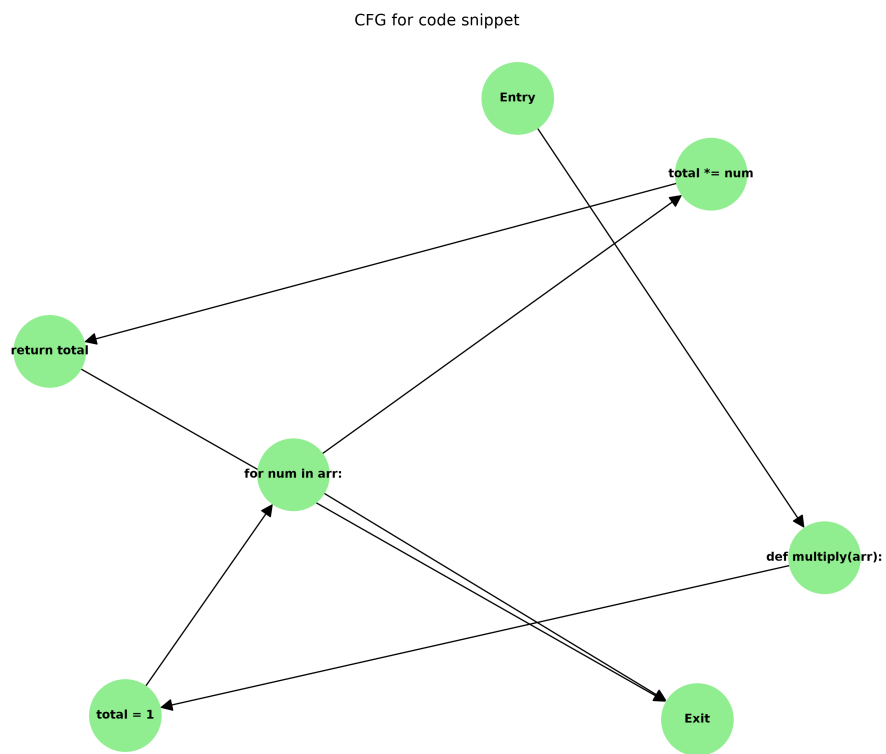


Figure 6: CFG for a function with loop

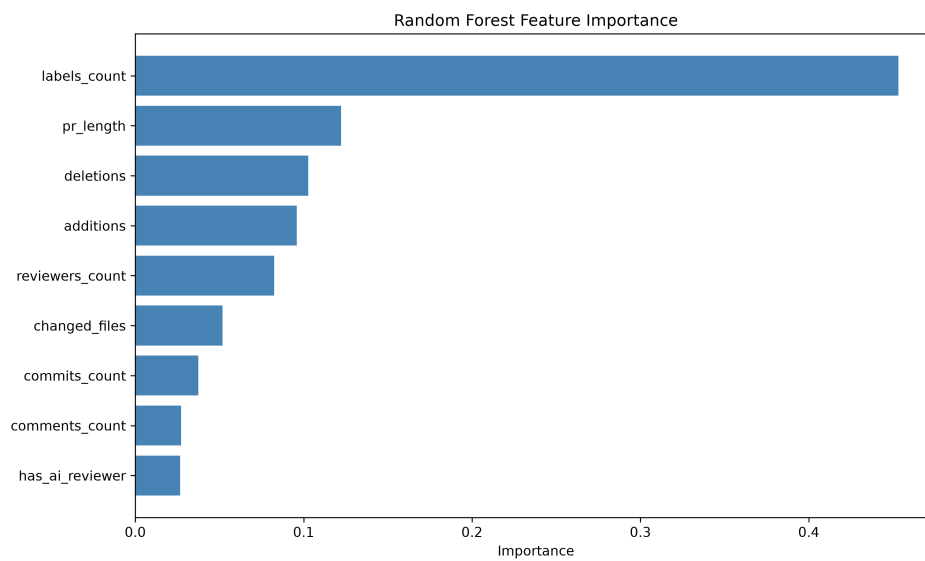


Figure 7: Random Forest feature importance

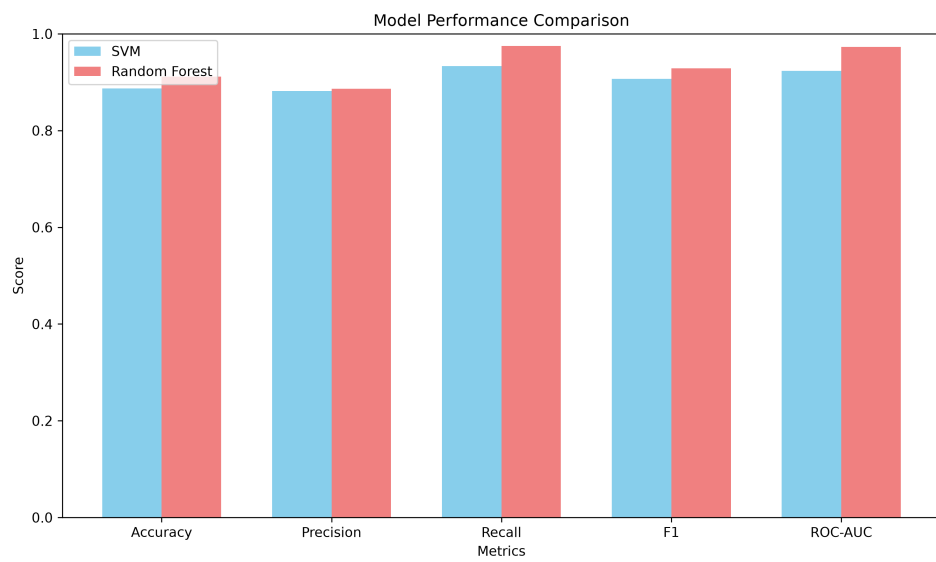


Figure 8: SVM vs Random Forest performance comparison

4. Summary of Problems Encountered

- Pandas and scikit-learn missing in venv - installed via pip. - AST/CFG visualization required networkx - installed separately. - sklearn package name deprecated - used scikit-learn instead.

5. Reflections on the Experiment

- AST/CFG capture program semantics that raw text cannot represent, enabling deeper code understanding. - Traditional ML requires manual feature engineering because algorithms cannot directly interpret code syntax and semantics. - SVM works well for small datasets with clear separation; Random Forest handles complex interactions and provides feature importance. - labels_count was the most important feature - PRs with more labels are better organized and more likely to merge. - Experiment 2 added feature engineering, scaling, model training, and evaluation to the data collection pipeline.

Instructor's Comments: Date: